

JAVA

Polimorfismo

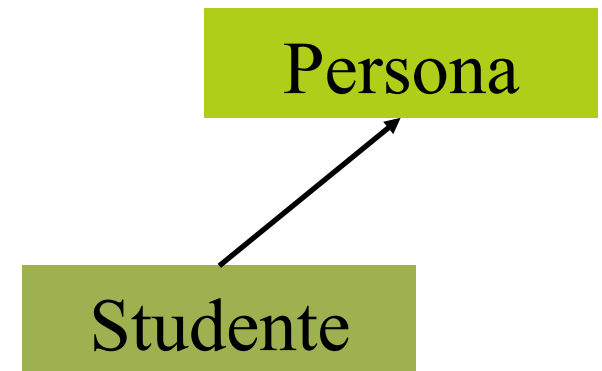
Classe Persona

Persona

```
public class Persona {  
    protected String nome;  
    protected int anni;  
    public Persona() {nome = "SCONOSCIUTO"; anni = 0; }  
    public Persona(String n) {nome = n; anni = 0; }  
    public Persona(String n, int a) {nome=n; anni=a; }  
    public void print() {  
        System.out.print("Mi chiamo " + nome);  
        System.out.println(" e ho " + anni + "anni");  
    }  
}
```

Classe Studente

```
public class Studente extends Persona {  
    protected int matr;  
    public Studente() {super(); matr = 9999; }  
    public Studente(String n) {  
        super(n); matr = 8888; }  
    public Studente(String n, int a) {  
        super(n,a); matr=7777; }  
    public Studente(String n, int a, int m) {  
        super(n,a); matr=m; }  
    public void print() {  
        super.print();  
        System.out.println("Matricola = " + matr);  
    }  
}
```



Classe EsempioDiCitta

```
public class EsempioDiCitta {
```

L'assegnamento $p=s$ non

PROBLEMA cosa stampa?

**p è un riferimento a Persona ma gli è stato
assegnato un oggetto Studente**

Se prevale la natura del riferimento, stamperà solo nome ed età

**Se prevale la natura dell'oggetto puntato,
stamperà nome, età e matricola**

È un problema di POLIMORFISMO

```
p.print(); // COSA STAMPA ???  
}
```

```
}
```

Polimorfismo

- Un metodo si dice polimorfo quando è in grado di adattare il suo comportamento allo specifico oggetto su cui deve operare.
- In Java, la possibilità di usare riferimenti a una data classe
 - ad esempio, Persona per puntare a oggetti di classi più specifiche
 - ad esempio, Studente
- introduce in astratto la possibilità di avere polimorfismo.
- In pratica, dipende cosa prevale:
 - se prevale il tipo del riferimento, non ci sarà mai polimorfismo e in tal caso, `p.print()` stamperà solo nome ed età, perché verrà invocato il metodo `print()` della classe Persona
 - se invece prevale il tipo dell'oggetto, allora ci potrà essere polimorfismo e in tal caso, `p.print()` stamperà nome, età e matricola, perché verrà invocato il metodo `print()` della classe Studente
- Java supporta il Polimorfismo e quindi prevale il tipo dell'oggetto

Binding statico vs dinamico

■ Binding statico

- le chiamate ai metodi sono collegate alla versione del metodo prestabilita a tempo di compilazione, basandosi sul tipo statico del riferimento. E' efficiente, ma non flessibile
- standard in C, default in C++, assente in Java

■ Binding dinamico

- le chiamate ai metodi sono collegate alla versione del metodo determinata a run-time, basandosi sul tipo dinamico dell'oggetto referenziato in quel momento. Un po' meno efficiente, ma molto flessibile
- non presente in C, possibile a richiesta in C++ (virtual), default in Java

Late binding - schema di principio

- Ogni istanza contiene un riferimento alla propria classe e include una tabella che mette in corrispondenza i nomi dei metodi da essa definiti con il codice compilato relativo a ogni metodo
- Chiamare un metodo comporta quindi:
 - accedere alla tabella opportuna in base alla classe dell'istanza
 - in base alla signature del metodo invocato, accedere alla entry della tabella corrispondente e ricavare il riferimento al codice del metodo
 - invocare il corpo del metodo così identificato.

A classe EsempioDiCittà

```
public class EsempioDiCitta {  
    public static void main(String args[]){
```

**void print() è un metodo polimorfo
poiché p referencia uno Studente,
stampa nome, età e matricola**

```
        p.print(), // COSA STAMPA ...  
    }  
}
```


Esempio

- Una piccola classe (eredita implicitamente il metodo toString() da Object):

```
public class Deposito {  
    float soldi;  
    public Deposito() { soldi=0; }  
    public Deposito(float s) { soldi=s; }  
}
```

Esempio

```
public class Esempio {  
    public static void main(String args[]){  
        Deposito d1 = new Deposito(312);  
        System.out.println(d1);  
    }  
}
```

Per stampare d1, viene invocato automaticamente
il metodo toString()

È una forma compatta per `System.out.println(d1.toString());`

Esempio

Se il toString() predefinito da Object non soddisfa, si può ridefinirlo:

```
public class Deposito {  
    float soldi;  
    public Deposito() { soldi=0; }  
    public Deposito(float s) { soldi=s; }  
    public String toString() {  
        return "Deposito di valore " + soldi;  
    }  
}
```

Viene creato un nuovo oggetto String concatenando la frase “Deposito di valore” con il risultato di Float.toString(soldi)

ESEMPIO

- L'output nel primo caso...

- Deposito@712c1a3c

**Identificativo univoco generato da Java:
nome della classe + hashing oggetto**

- ... e nel secondo caso:

- Deposito di valore 312.0

Esempio completo

```
public abstract class Animale {
    private String nome;
    protected String verso;
    public Animale(String s) { nome=s; }
    public abstract String si_muove();
    public abstract String vive();
    public abstract String chi_sei();
    public abstract String verso();
    public void mostra() {
        System.out.println(nome + ", " +
            chi_sei() + ", " + verso + ", si muove " +
            si_muove() + " e vive " + vive() );
    }
}
```

Output

Tweet tweet flap flap

Sniff sniff woof woof

Glug glug gurgle gurgle

```
public class Zoo {  
  
    public static void main (String[ ] argv) {  
  
        Animale[ ] AnimaliArray = new Animale[3];  
        int index;  
  
        AnimaliArray[0] = new Uccello( );  
        AnimaliArray[1] = new Cane( );  
        AnimaliArray[2] = new Pesce( );  
  
        for (index = 0; index < AnimaliArray.length; index++){  
            AnimaliArray[index].verso( );  
        }  
    } // end del main  
} // end classe prova
```

La classe Animale ha il metodo verso() così ogni membro della classe può fare un verso.

Riassumendo...

- *Polimorfismo significa “prendere molte forme” ... un riferimento a una data classe puo' prendere la forma di ognuna delle sue sottoclassi.*
- Polimorfismo e Legame Dinamico (Dynamic Binding) insieme assicurano il corretto funzionamento del metodo verso() dell'esempio precedente.
- Un oggetto di una sottoclasse puo sostituire un oggetto della superclasse: “Un uccello e' un Animale”
- Il contrario non e' vero: non si puo sostituire un elemento di una sottoclasse con uno della superclass “Un Animale non e' un uccello”.
- *Abbiamo una singola interfaccia per un comportamento multiplo:*
 - Solo una interfaccia per la chiamata del metodo.
- Comportamento multiplo basato sulla sottoclasse

```
public class Esempio2 {  
    public static void main(String[ ] argv) {  
        Animale AnimaliArray [ ] = new Animale[3];  
        Cane c;  
        int i;  
        AnimaliArray[0] = new Uccello( );  
        AnimaliArray[1] = new Cane( );  
        AnimaliArray[2] = new Pesce( );  
        for (i = 0; i < AnimaliArray.length; i++){  
            AnimaliArray[i].verso();  
            if (AnimaliArray[i] instanceof Cane){  
                c = (Cane) AnimaliArray[i];  
                c.ringhiare( );  
            }  
        }  
    } // main  
} // Esempio2
```

Noi possiamo chiamare
il metodo ringhiare()
Poiché solo il cane
ringhia, gli altri Animali
non hanno questo
metodo.

Polimorfismo: casting

Casting:

- usato qui per dare ad un oggetto di una sottoclasse la forma della sottoclasse appropriata per consentire la chiamata del metodo, e.g.,

```
if (AnimaliArray[i] instanceof Cane) {  
    AnimaliArray[i].ringhiare();  
}
```

Dovrebbe produrre un errore perchè un oggetto della classe *Animale* non ha il metodo *ringhiare()*.

Così, noi prima eseguiamo il cast dell'oggetto:

```
if (AnimaliArray[i] instanceof Cane) {  
    c = (Cane) AnimaliArray[i]  
    c.ringhiare();  
}
```

Polimorfismo: casting

Keyword *instanceof*:

E' utilizzata per richiedere ad un oggetto di determinare se è una istanza di una specifica classe, ad esempio

“E' un particolare Animale della classe cane?”

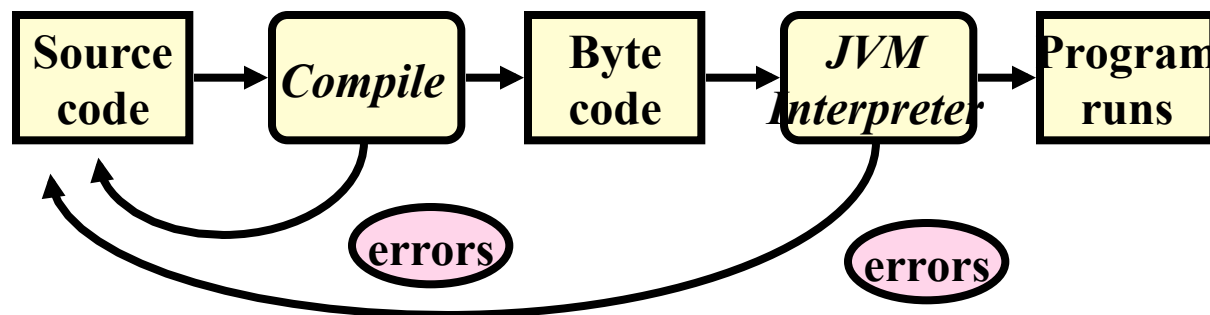
Problema:

Se Java puo determinare cos'e' (o non è) un dato oggetto attraverso l'uso di *instanceof* allora:

- perchè è necessario il cast?
- perchè Java non può fare questo per noi?

Polimorfismo: casting

differenze fra *compile-time* e *run-time* type checking.



Errori a Compile-time:

- Quelli che sono rilevabili senza che il programma sia un'esecuzione.

```
int index
string strName
index = strName;
```

Istruzione illegale

Errori a Run-time:

- Quelli che sono riconoscibili solo durante l'esecuzione con i valori reali.

```
AnimaliArray[<indice>] = UnAnimale
```

L'istruzione è corretta ma può non essere corretta per alcuni valori di indice

Polimorfismo: casting

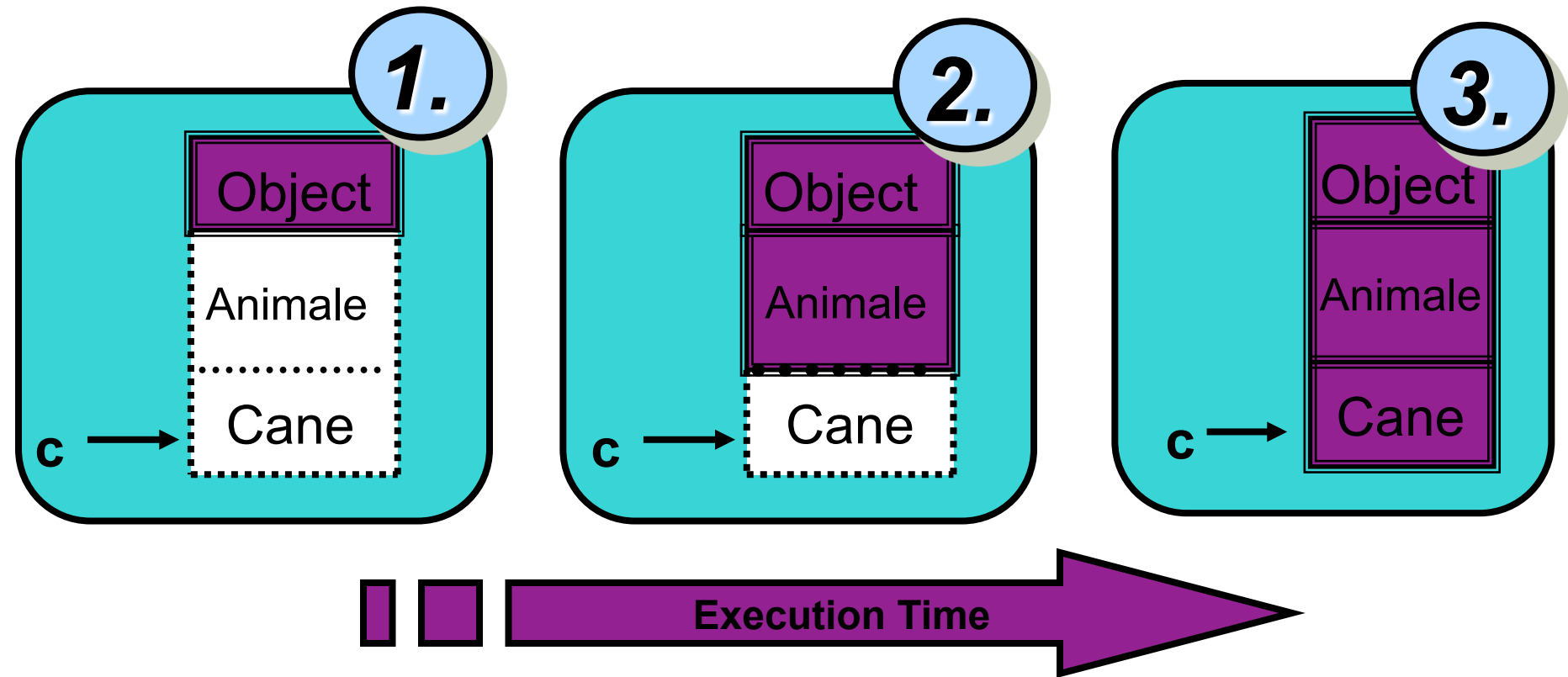
```
if (AnimaliArray[i] instanceof Cane) {  
    AnimaliArray[i].ringhiare();  
}
```

- La prima riga è corretta.
- La seconda non è corretta (tranne che l'array non sia dichiarato di elementi Cane)
- Compilatore non può vedere il legame fra le istruzioni
- Runtime system potrebbe
- MA. . . Per garantire performance vogliamo effettuare queste verifiche durante la compilazione

```
if (AnimaliArray[i] instanceof Cane) {  
    c = (Cane) AnimaliArray[i];  
    c.ringhiare();  
}
```

Come sono creati gli oggetti

```
Cane c = new Cane();
```

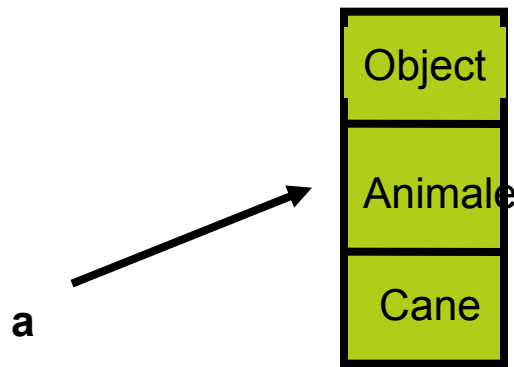


Un chiamata implicita `super()` richiama le classi padre.
In conclusione un `Cane` e' un `Animale`, un `Animale` è un `Object`

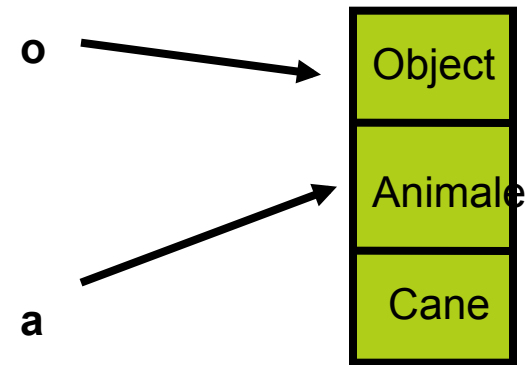
Polimorfismo

```
Animale a = new Cane();
```

Noi possiamo creare un nuovo riferimento che punta a tipi
differente nello stesso blocco di memoria.



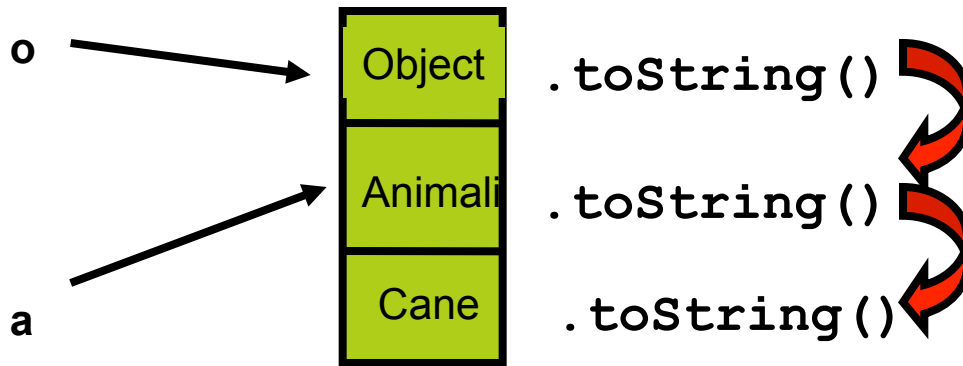
```
Animale a = new Cane();
```



```
Object o = a;
```

In un certo senso, il casting è una sorta di polimorfismo.

Dynamic Binding

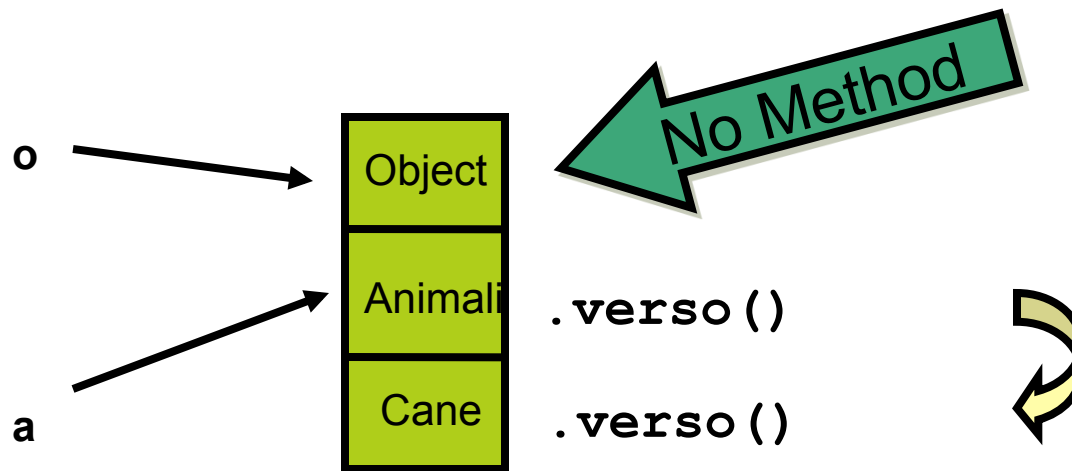


```
System.out.println(o.toString());
```

Quando chiamiamo un metodo su un riferimento, il metodo deve esistere (o essere ereditato) nel tipo. Comunque, la specifica implementazione è determinata a run-time. Cioè viene utilizzato il **'dynamic binding'**.

Dynamic binding fornisce una soluzione a runtime fra le diverse implementazioni

Casting e polimorfismo

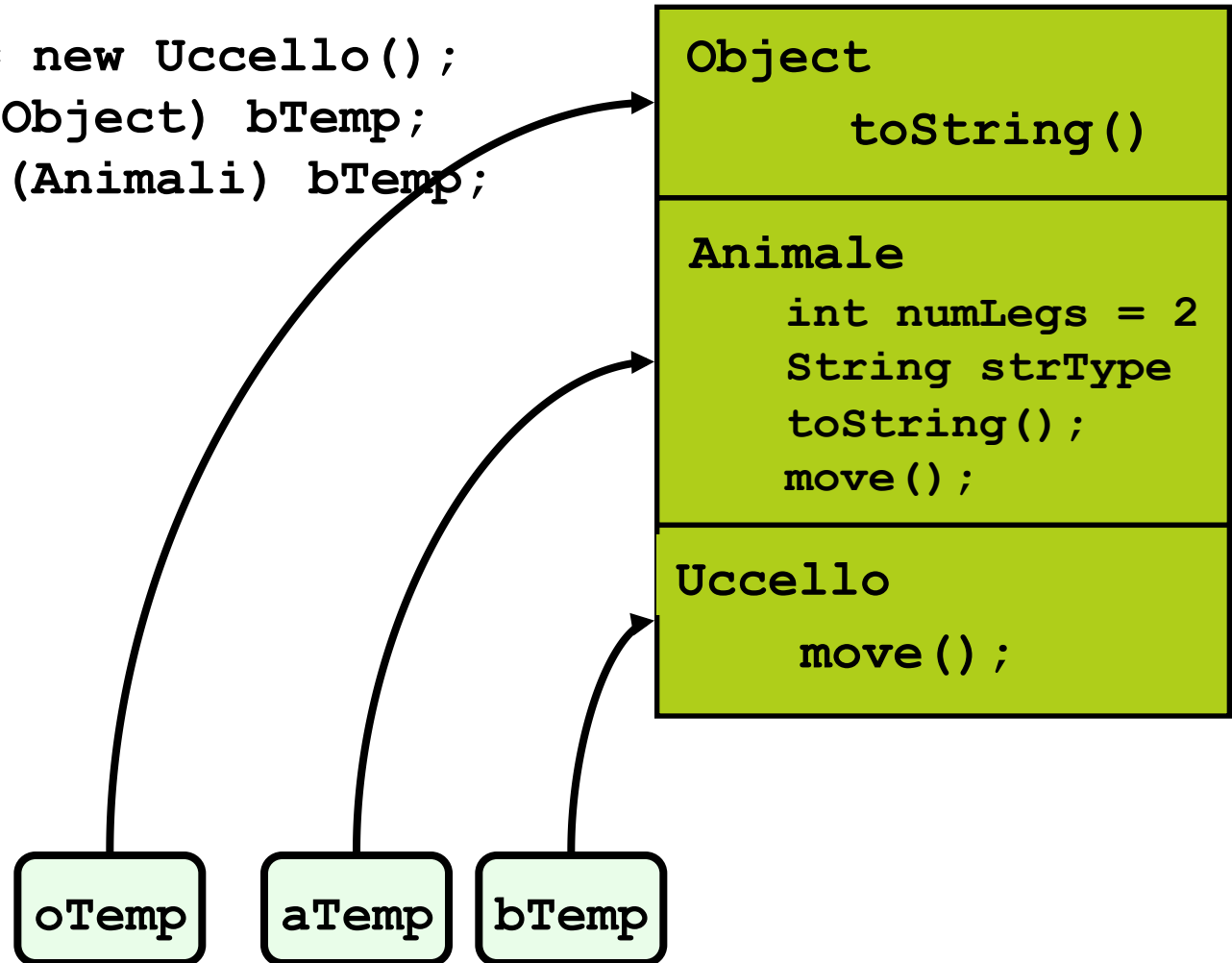


Il dynamic binding non fa miracoli. Il tipo deve avere il metodo disponibile (nella classe corrente e nella sua superclasse) altrimenti si ha un **errore di compilazione**.

```
o.verso(); // COMPILER ERROR!
```


Polimorfismo

```
Uccello bTemp = new Uccello();  
Object oTemp = (Object) bTemp;  
Animale aTemp = (Animali) bTemp;
```



Object

`toString();`

Animali []

`final int length = 3`

[0]

[1]

[2]

Object

`toString();`

Animale

`int numLegs = 2
String strType
toString();
move();`

Uccello

`move();`

Object

`toString();`

Animale

`int numLegs = 0
String strType
toString();
move();`

Pesce

`move();`

Object

`toString();`

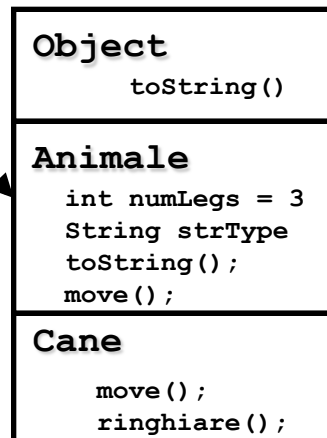
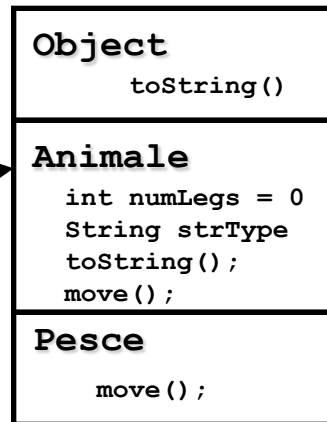
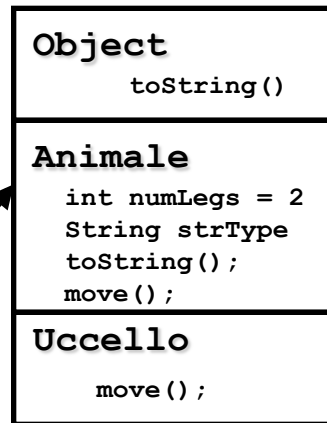
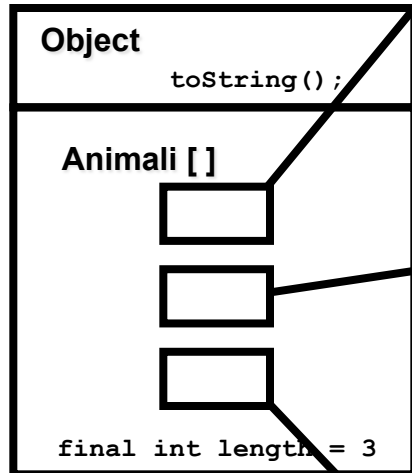
Animale

`int numLegs = 3
String strType
toString();
move();`

Cane

`move();
ringhiare();`

E' corretto?



Cosa accade se eseguiamo il codice

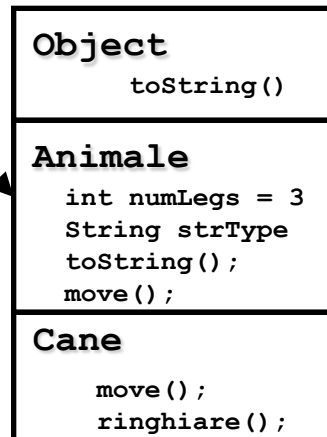
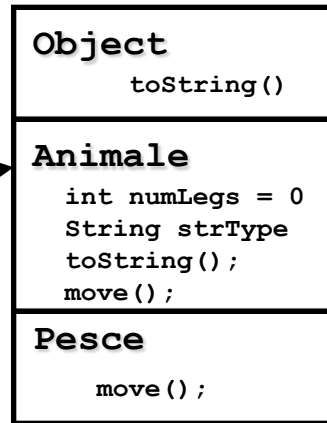
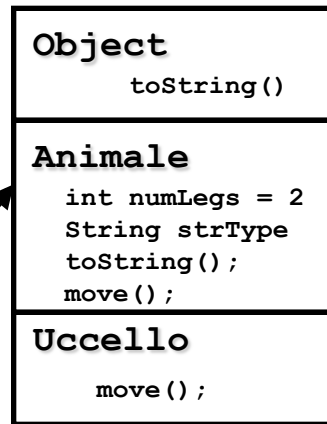
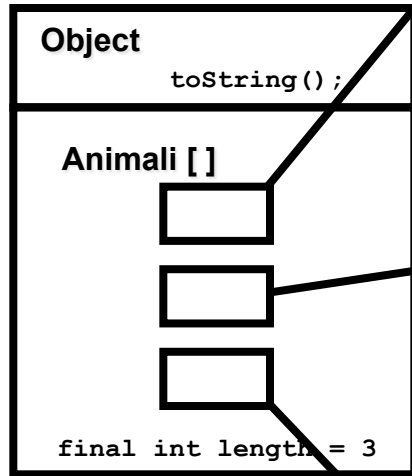
```
Object oTemp;  
oTemp = AnimaliArray[1];  
oTemp.move();
```

Object

`oTemp`

No. Il codice non funziona.

E' corretto?

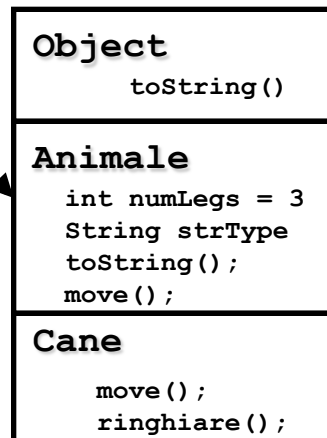
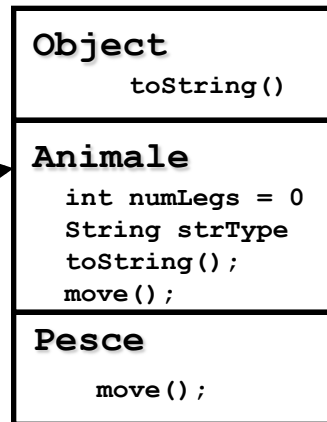
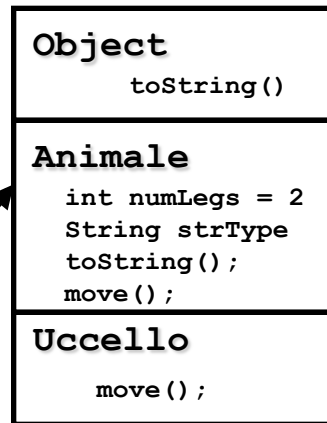
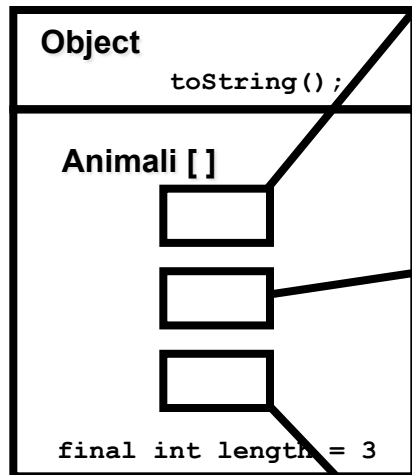


Compile time

```
Object oTemp;  
oTemp = AnimaliArray[1];  
oTemp.move();
```

oTemp

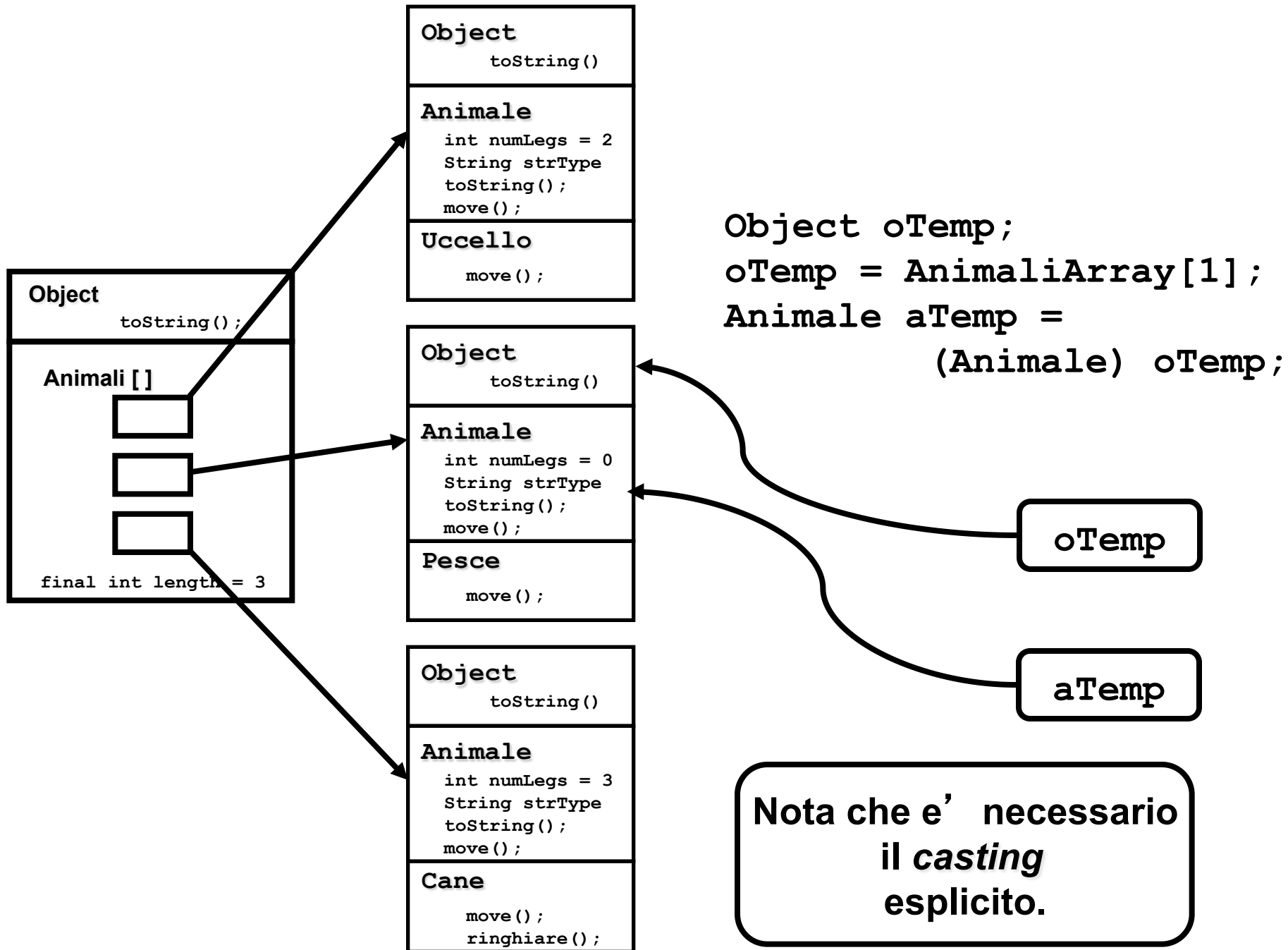
E' corretto?

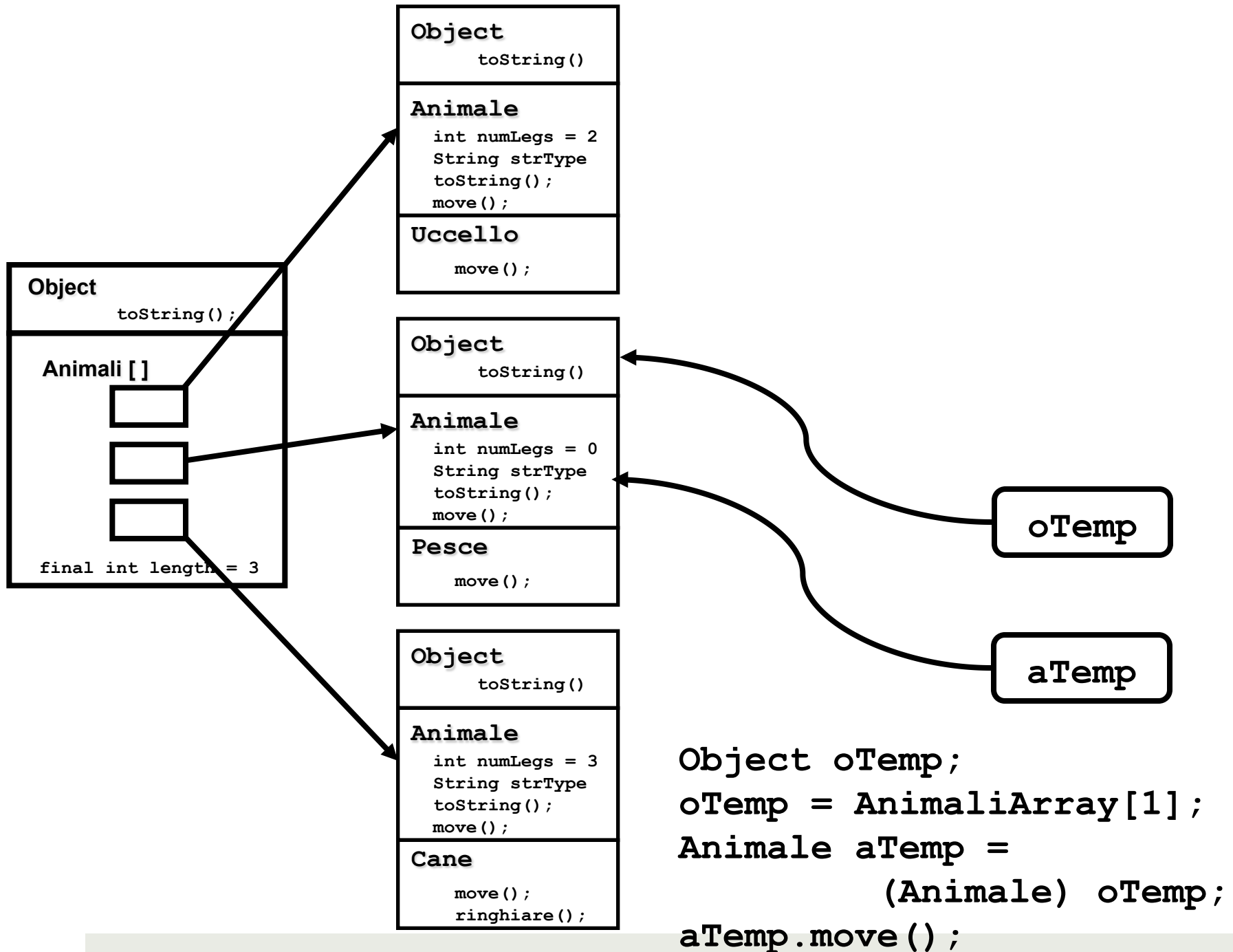


```
Object oTemp;  
oTemp = AnimaliArray[1];  
oTemp.move();
```

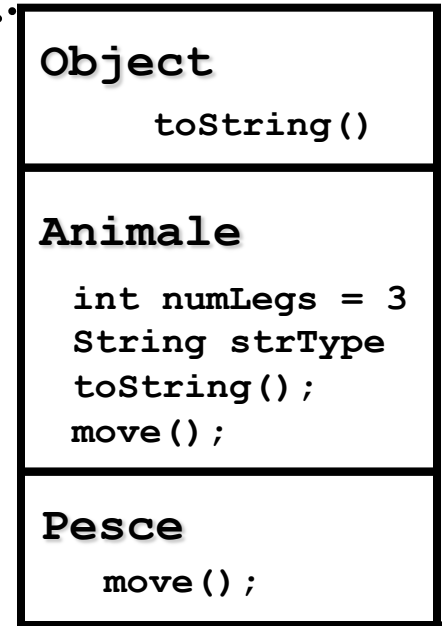
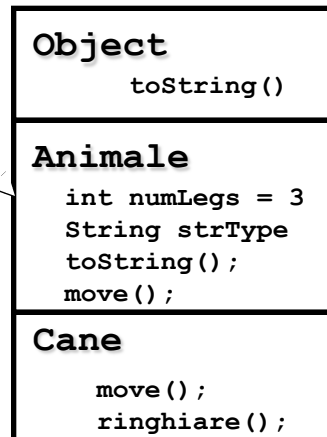
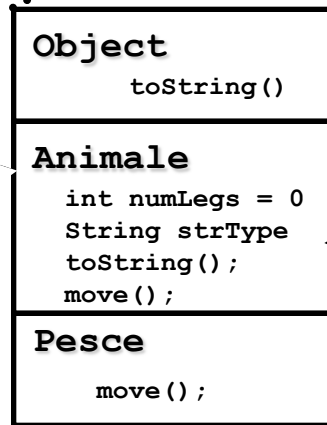
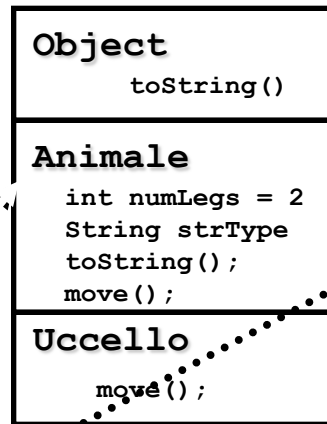
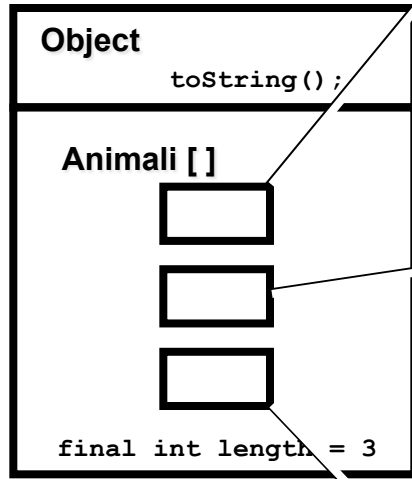
oTemp

NO.
La classe Object
non ha il metodo
move()





```
Object oTemp;
oTemp = AnimaliArray[1];
Animale aTemp =
    (Animale) oTemp;
aTemp.move();
```



oTemp

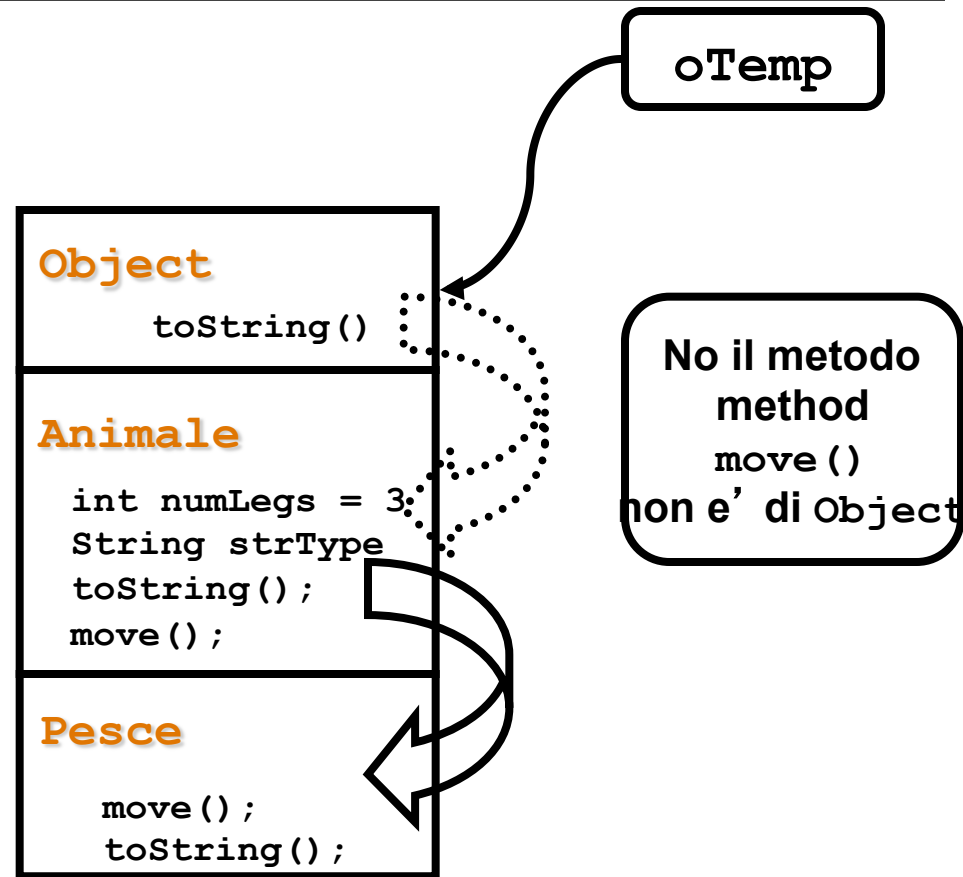
aTemp

Sia Animale che
Pesce
hanno il metodo
move() .

Il tipo del riferimento
determina se abbiamo
Animale
o Pesce

“Always?”

```
Object oTemp;  
oTemp = AnimaliArray[1];  
oTemp.move(); // sbagliato!
```



NO. Questo puo' causare un errore di compilazione. Java e' “**strongly typed**”, cioe ogni volta che invochi un metodo il metodo deve essere presente nella classe ma il dynamic binding consente di utilizzare una versione del metodo piu specifica.

Metodi override

- Come è noto un punto di forza di Java è dato dall'*ereditarietà*, per poterla ben gestire a volte risulta utile **annotare l'override dei metodi**.

Nel particolare la clausola **@Override**, anteposta ad un determinato elemento indica che il **componente** in oggetto sovrascrive l'elemento genitore da cui eredita.

Ok

```
class Classe1 {  
    void m1() {  
        System.out.println("Metodo m1 della classe1");  
    }  
class Classe2 extends Classe1 {  
    @Override  
    void m1() {  
        System.out.println("Metodo m1 della classe2");  
    }  
}
```

Errore in compilazione

```
class Classe1 {  
    void m1() {  
        System.out.println("Metodo m1 della classe1");  
    }  
class Classe2 extends Classe1 {  
    @Override  
    void m () {  
        System.out.println("Metodo m1 della classe2");  
    }  
}
```